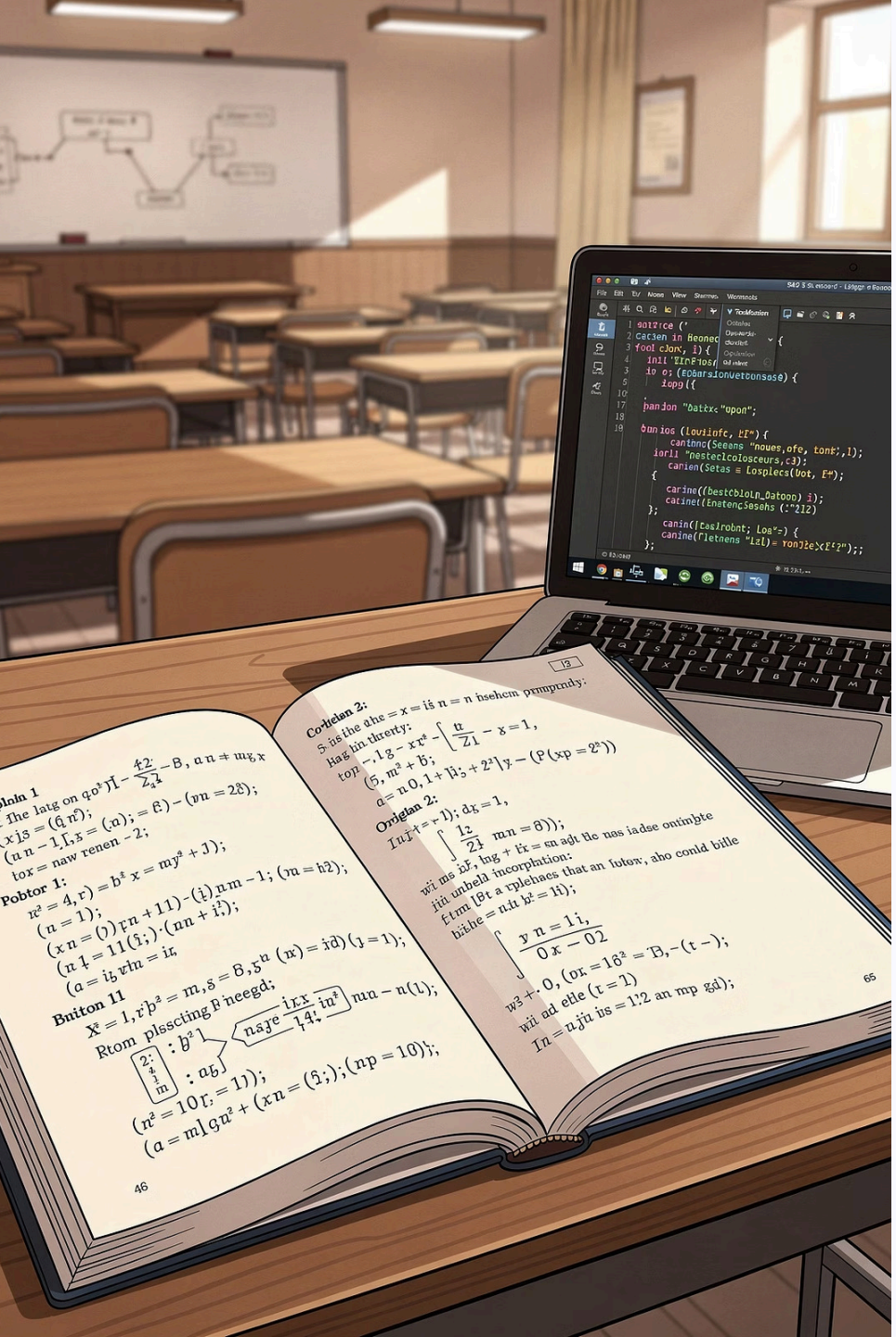


Chapter 6: List Data Structures

Course 040223101 · Computer Programming for Mathematics 1
Department of Mathematics, Faculty of Applied Science, King Mongkut's University of Technology North Bangkok

Instructor: Assoc. Prof. Dr. Anuchit Jitpattanakul · Semester 1, Academic Year 2026

Learning Objectives

Upon completing this chapter, students will be able to:

01

Create Lists & Access Members

Build lists and access elements using both positive and negative indices.

02

Use Slicing

Extract sub-ranges of a list using the `list[start:stop:step]` syntax.

03

Apply List Methods

Use built-in methods such as `append`, `insert`, `remove`, and `sort`.

04

Iterate with for Loops

Loop over lists with `for` to process data element by element.

05

Apply Lists to Mathematics

Store and compute mathematical data sets using list structures.

This chapter aligns with Course Learning Outcome **CLO 4**.

Why Do We Need Lists?

So far, a single variable stores only one value. In real-world applications, however, we often need to manage large collections of data – for example, the scores of every student in a class.

A **list** is a data structure that stores multiple values in a single variable in an ordered sequence. This chapter covers how to create, access, and manipulate lists in Python.

6.1 Creating a List

A list is created by placing members inside square brackets `[ ]`, separated by commas. Members can be of the same type or different types.

Syntax

`my_list = [value1, value2, value3, ...]`

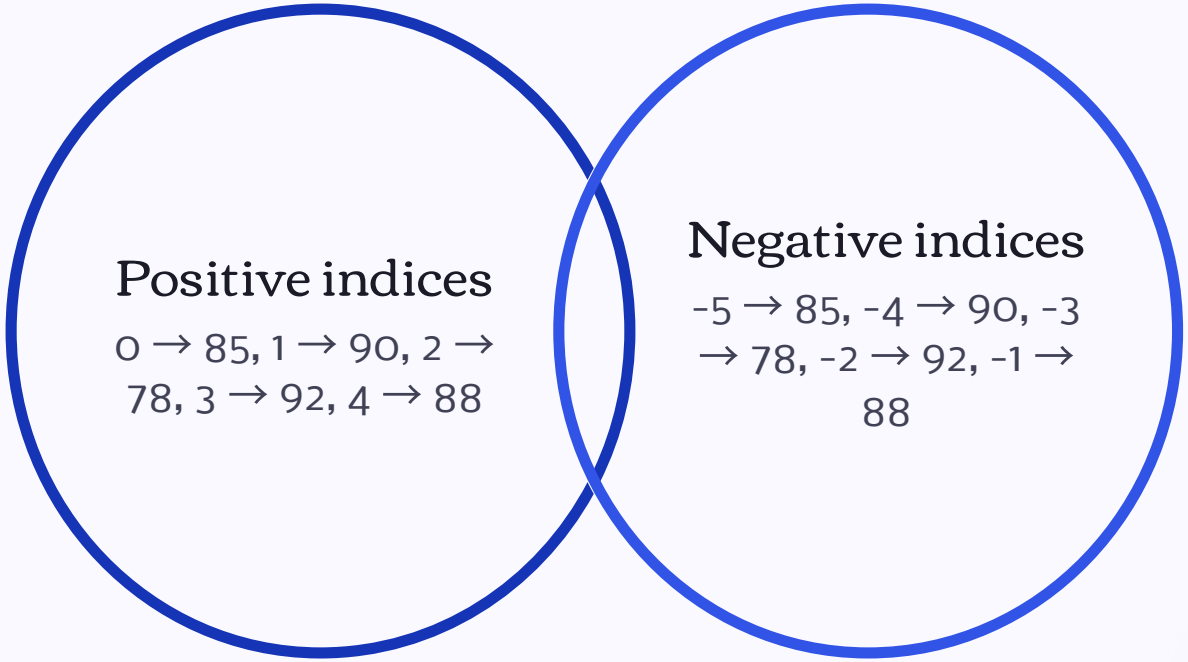
Members may be integers, floats, strings, or even other lists.

Examples

```
scores = [85, 90, 78, 92, 88]
names = ["Alice", "Bob", "Carol"]
mixed = [1, "hello", 3.14, True]
empty = []
```

Accessing List Members by Index

Each member in a list has an **index** (position number). Python indices start at **0**. Negative indices count from the end of the list.



Positive indices start at 0 (leftmost element); negative indices start at −1 (rightmost element), making it easy to access the last element without knowing the list length.

Index Access – Code Example

File: o6list1.py

```
scores = [85, 90, 78, 92, 88]

print(scores[0])   # First member
print(scores[2])   # Third member
print(scores[-1])  # Last member
print(len(scores)) # Number of members
```

Output

```
85
78
88
5
```

Explanation

- `scores[0]` → 85 (first element)
- `scores[2]` → 78 (third element)
- `scores[-1]` → 88 (last element)
- `len(scores)` → 5 (total count)

IndexError Warning

⊗ If you reference an index that is out of bounds, Python will raise an **IndexError**. For example, `scores[5]` on a list with 5 members (valid indices: 0 - 4) will crash the program.

Valid Indices

For a list of length `n`, valid positive indices are `0` to `n-1`, and valid negative indices are `-1` to `-n`.

Common Mistake

Beginners often try `list[n]` expecting the last element, but the last valid index is `n-1`. Use `list[-1]` instead.

Safe Access

Always check `len(list)` before accessing by index, or use `try/except IndexError` to handle the error gracefully.

6.2 Slicing

Slicing extracts a sub-range of elements from a list. The syntax is:

```
list[start : stop : step]
```

The result includes elements from index `start` up to (but **not including**) index `stop`. The optional `step` controls how many positions to skip between each selected element.

Slicing – Code Example

File: o6slice.py

```
nums = [10, 20, 30, 40, 50, 60]

print(nums[1:4])  # Indices 1 to 3
print(nums[:3])   # From start to index 2
print(nums[3:])   # From index 3 to end
print(nums[::2])  # Every other element
```

Output

```
[20, 30, 40]
[10, 20, 30]
[40, 50, 60]
[10, 30, 50]
```

Notes

- Omitting `start` defaults to 0
- Omitting `stop` defaults to end of list
- `step=2` skips every other element
- Slicing never raises `IndexError`

Slicing Quick Reference

`lst[a:b]`

Elements from index `a` to `b−1`. The most common form.

`lst[:b]`

Elements from the beginning up to index `b−1`. Equivalent to `lst[0:b]`.

`lst[a:]`

Elements from index `a` to the end of the list.

`lst[::k]`

Every `k`-th element across the entire list. Use `k=−1` to reverse.

6.3 Modifying Lists – Mutability

Unlike strings, lists are **mutable** – their contents can be changed after creation. You can reassign individual elements, add new elements, or remove existing ones.

Reassign an Element

```
scores[0] = 95
```

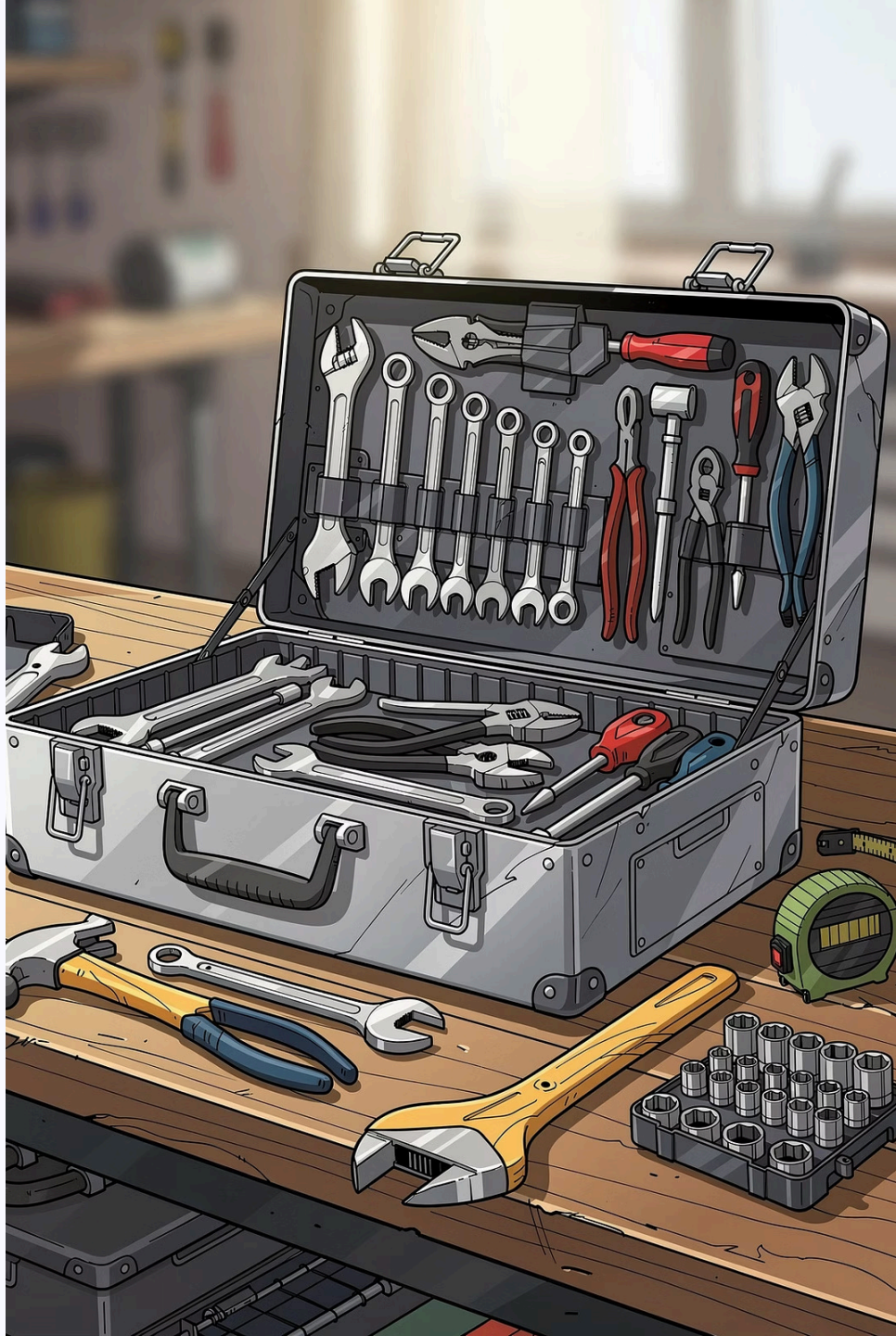
Directly assign a new value to any index.

Add Elements

Use `append()` to add to the end, or `insert()` to add at a specific position.

Remove Elements

Use `remove()` to delete by value, or `pop()` to delete by index and retrieve the value.



Important List Methods

Method	Description	Example
`lst.append(x)`	Add `x` to the end of the list	`lst.append(99)`
`lst.insert(i, x)`	Insert `x` at index `i`	`lst.insert(1, "mango")`
`lst.remove(x)`	Remove the first occurrence of `x`	`lst.remove("banana")`
`lst.pop(i)`	Remove and return element at index `i`	`val = lst.pop(0)`
`lst.sort()`	Sort elements in ascending order (in-place)	`lst.sort()`
`lst.reverse()`	Reverse the order of elements (in-place)	`lst.reverse()`
`lst.index(x)`	Return the index of the first occurrence of `x`	`lst.index("apple")`
`lst.count(x)`	Count how many times `x` appears	`lst.count(5)`

List Methods – Code Example

File: o6method.py

```
fruits = ["apple", "banana"]
fruits.append("cherry")
fruits.insert(1, "mango")
print(fruits)

fruits.remove("banana")
print(fruits)

nums = [5, 2, 9, 1]
nums.sort()
print(nums)
```

Output

```
['apple', 'mango', 'banana', 'cherry']
['apple', 'mango', 'cherry']
[1, 2, 5, 9]
```

Step-by-Step

1. Start with `["apple", "banana"]`
2. `append("cherry")` adds to end
3. `insert(1, "mango")` inserts at index 1
4. `remove("banana")` deletes first match
5. `sort()` arranges numerically ascending

append() vs insert()

append(x)

Always adds the new element to the **end** of the list. It is the fastest and most common way to grow a list.

```
lst = [1, 2, 3]
lst.append(4)
# [1, 2, 3, 4]
```

insert(i, x)

Inserts `x` **before** the element currently at index `i`. All subsequent elements shift one position to the right.

```
lst = [1, 2, 3]
lst.insert(1, 99)
# [1, 99, 2, 3]
```

Use `append()` when order doesn't matter or you're building a list incrementally. Use `insert()` when position is important.

remove() vs pop()

remove(x) – Delete by Value

Searches the list for the **first occurrence** of value `x` and removes it. Raises `ValueError` if `x` is not found. Does not return the removed value.

```
lst = ["a", "b", "a"]
lst.remove("a")
# ["b", "a"]
```

pop(i) – Delete by Index

Removes the element at index `i` and **returns it**. If no index is given, removes and returns the last element. Raises `IndexError` if index is out of range.

```
lst = [10, 20, 30]
val = lst.pop(1)
# val = 20, lst = [10, 30]
```

6.4 Iterating Over Lists with for Loops

A `for` loop can iterate directly over a list. On each iteration, the loop variable receives the value of the next element. This is the most Pythonic way to process every element in a list.

Direct Iteration

```
for element in my_list:
    # process element
```

The loop variable takes each value in sequence.

Index-Based Iteration

```
for i in range(len(my_list)):
    # use my_list[i]
```

Use when you need the index position as well as the value.

Loop Example – Sum and Average

File: o6loop.py

```
scores = [85, 90, 78, 92, 88]
total = 0

for s in scores:
    total += s

avg = total / len(scores)
print(f"Sum = {total}, Average = {avg:.2f}")
```

Output

```
Sum = 433, Average = 86.60
```

How It Works

1. Initialize `total = 0`
2. Each iteration adds the current score `s` to `total`
3. After the loop, divide by `len(scores)` to get the average
4. Format to 2 decimal places with `:.2f`

Built-in Functions for Numeric Lists

Python provides ready-made functions that work directly on lists of numbers, saving you from writing manual loops for common operations.

sum()

Total Sum

Returns the sum of all elements. `sum(scores)` → 433

max()

Maximum Value

Returns the largest element. `max(scores)` → 92

min()

Minimum Value

Returns the smallest element. `min(scores)` → 78

len()

Count of Elements

Returns the number of elements. `len(scores)` → 5

Built-in Functions – Code Example

File: o6builtin.py

```
scores = [85, 90, 78, 92, 88]

print("Sum:    ", sum(scores))
print("Max:    ", max(scores))
print("Min:    ", min(scores))
print("Average:", sum(scores) / len(scores))
```

Output

```
Sum:     433
Max:     92
Min:     78
Average: 86.6
```

Why Use Built-ins?

These functions are implemented in optimized C code internally, making them faster and more readable than equivalent manual loops. Always prefer built-ins when available.

6.5 Applied Example: Statistical Calculations

This section combines list knowledge to compute the **mean** and **standard deviation** of a data set – two of the most fundamental statistics in mathematics.

Mean (Average)

Sum all values and divide by the count: $\bar{x} = \frac{\sum x_i}{n}$

Variance

Average of squared deviations from the mean: $\sigma^2 = \frac{\sum (x_i - \bar{x})^2}{n}$

Standard Deviation

Square root of variance: $\sigma = \sqrt{\sigma^2}$

Statistics Code – 06stat.py

```
data = [4, 8, 15, 16, 23, 42]
n    = len(data)
mean = sum(data) / n

var = 0
for x in data:
    var += (x - mean) ** 2
var = var / n

sd = var ** 0.5

print(f"Mean              = {mean:.2f}")
print(f"Standard Deviation = {sd:.2f}")
```

Output

```
Mean              = 18.00
Standard Deviation = 12.32
```

Algorithm Walkthrough

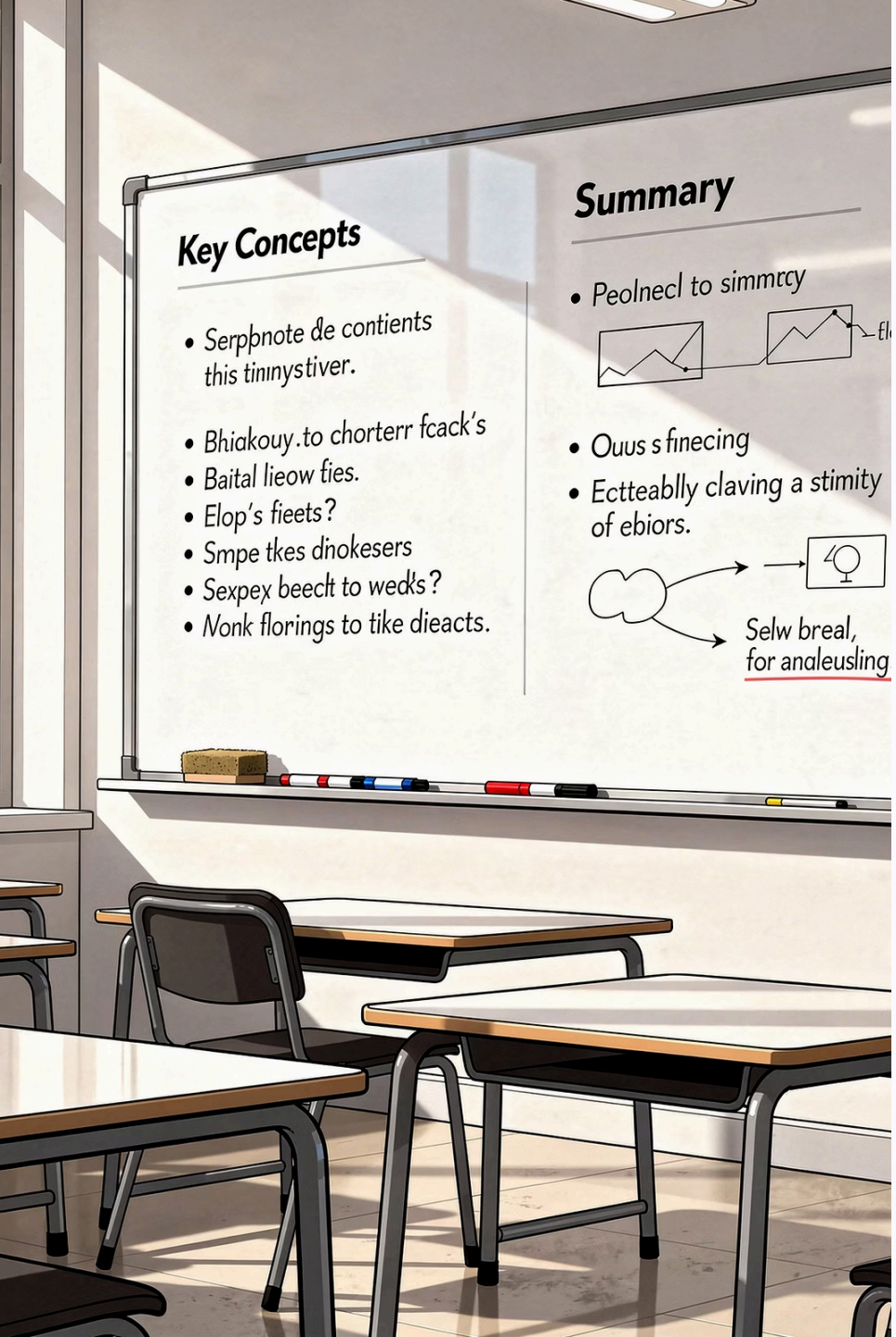
1. Compute `mean` using `sum()` and `len()`
2. Loop through each value `x`, accumulate $(x - \text{mean})^2$
3. Divide accumulated sum by `n` to get variance
4. Take `var ** 0.5` (square root) for standard deviation

Tracing Through the Statistics Example

Data set: `[4, 8, 15, 16, 23, 42]` – Mean = 18.00

x	x – mean	(x – mean)²	Running var sum	Step
4	−14	196	196	1
8	−10	100	296	2
15	−3	9	305	3
16	−2	4	309	4
23	+5	25	334	5
42	+24	576	910	6

Variance = 910 / 6 ≈ 151.67 → Standard Deviation = $\sqrt{151.67} \approx$ **12.32**



Chapter 6 Summary

Lists Store Multiple Values

A single list variable holds many values in an ordered sequence, replacing the need for many separate variables.

Index & Slicing

Access elements with indices (starting at 0; negative indices count from the end). Extract ranges with `list[start:stop:step]`.

Methods & Mutability

Lists are mutable. Use `append`, `insert`, `remove`, `pop`, `sort`, `reverse`, `index`, `count` to modify and query lists.

Loops & Built-ins

Iterate with `for`. Use `sum()`, `max()`, `min()`, `len()` for quick numeric computations.

Lists are the most frequently used data structure in mathematical data processing in Python.

End-of-Chapter Exercises

Apply your knowledge of lists with the following programming exercises:

1

Score Statistics

Create a list storing 6 scores, then display the maximum, minimum, and average.

2

Sort Descending

Write a program that accepts 5 numbers from the user, stores them in a list, and displays the list sorted from largest to smallest.

3

Count Even & Odd

Write a program that counts the number of even and odd numbers in the list `[12, 7, 5, 8, 20, 3, 14]`.

4

Squares List

Write a program that builds a list of squares of integers 1 through 10, then displays the sum of all squares.

Exercise 2 Solution – Sort Descending

Accept 5 numbers from the user, store in a list, and display sorted from largest to smallest.

```
nums = []
for i in range(5):
    nums.append(float(input(f"Number {i+1}: ")))

nums.sort(reverse=True)
print(nums)
```

Sample Output

```
Number 1: 3
Number 2: 9
Number 3: 5
Number 4: 20
Number 5: 8

[20.0, 9.0, 8.0, 5.0, 3.0]
```

Key Points

- Start with an empty list `nums = []`
- Use `append()` inside a loop to collect inputs
- `float(input(...))` converts string input to a number
- `sort(reverse=True)` sorts descending in-place

Exercise 3 Solution – Count Even & Odd

Count even and odd numbers in the list `[12, 7, 5, 8, 20, 3, 14]`.

```
nums = [12, 7, 5, 8, 20, 3, 14]
even = odd = 0

for x in nums:
    if x % 2 == 0:
        even += 1
    else:
        odd += 1

print(f"Even: {even}, Odd: {odd}")
```

Output

```
Even: 4, Odd: 3
```

Key Points

- The modulo operator `%` gives the remainder after division
- If `x % 2 == 0`, the number is even; otherwise odd
- Two counters `even` and `odd` are initialized to o
- Each element is tested and the appropriate counter incremented

Exercise 4 Solution – List of Squares

Build a list of squares of integers 1 through 10, then display the sum.

```
squares = []
for i in range(1, 11):
    squares.append(i ** 2)

print(squares)
print("Sum =", sum(squares))
```

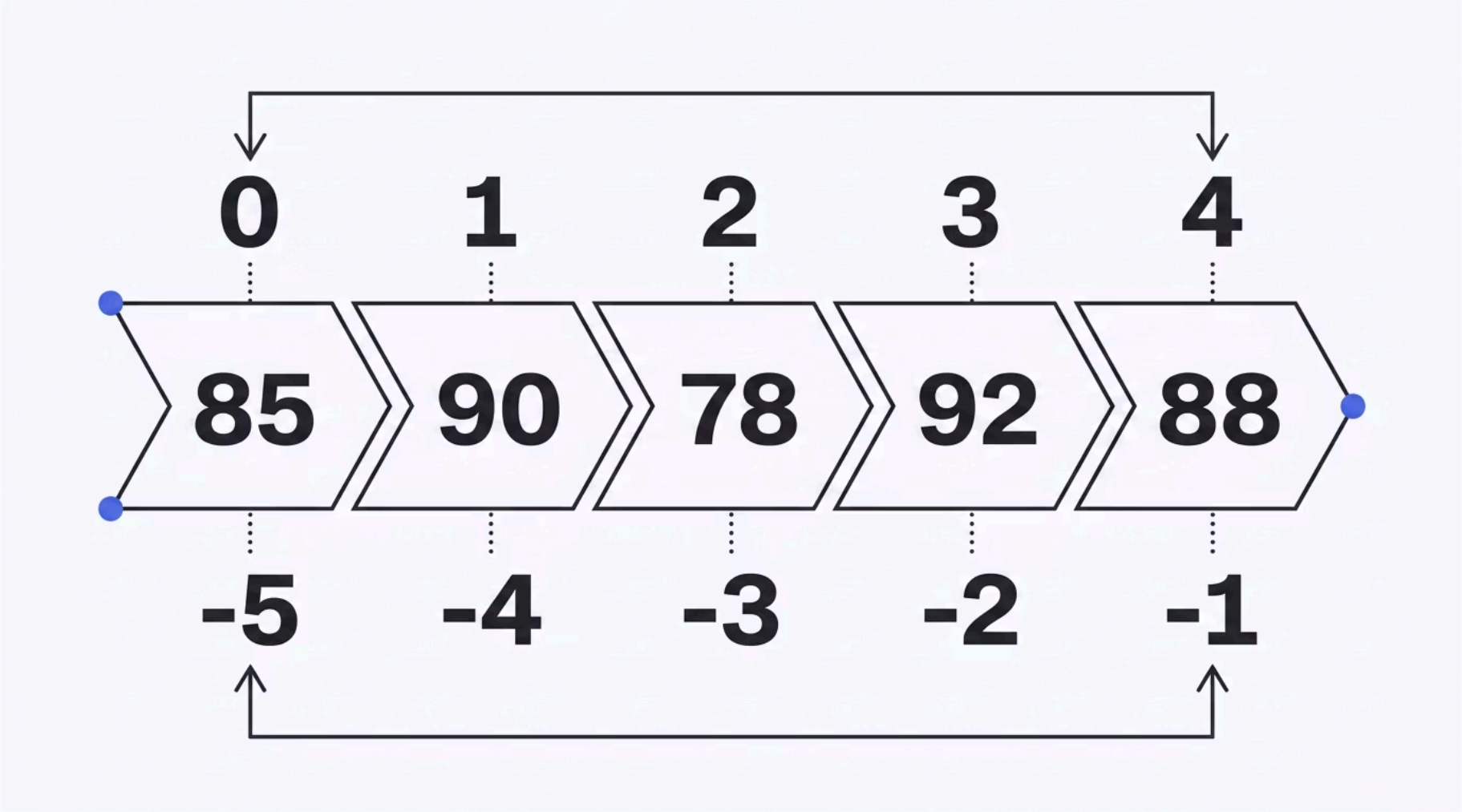
Output

```
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
Sum = 385
```

Key Points

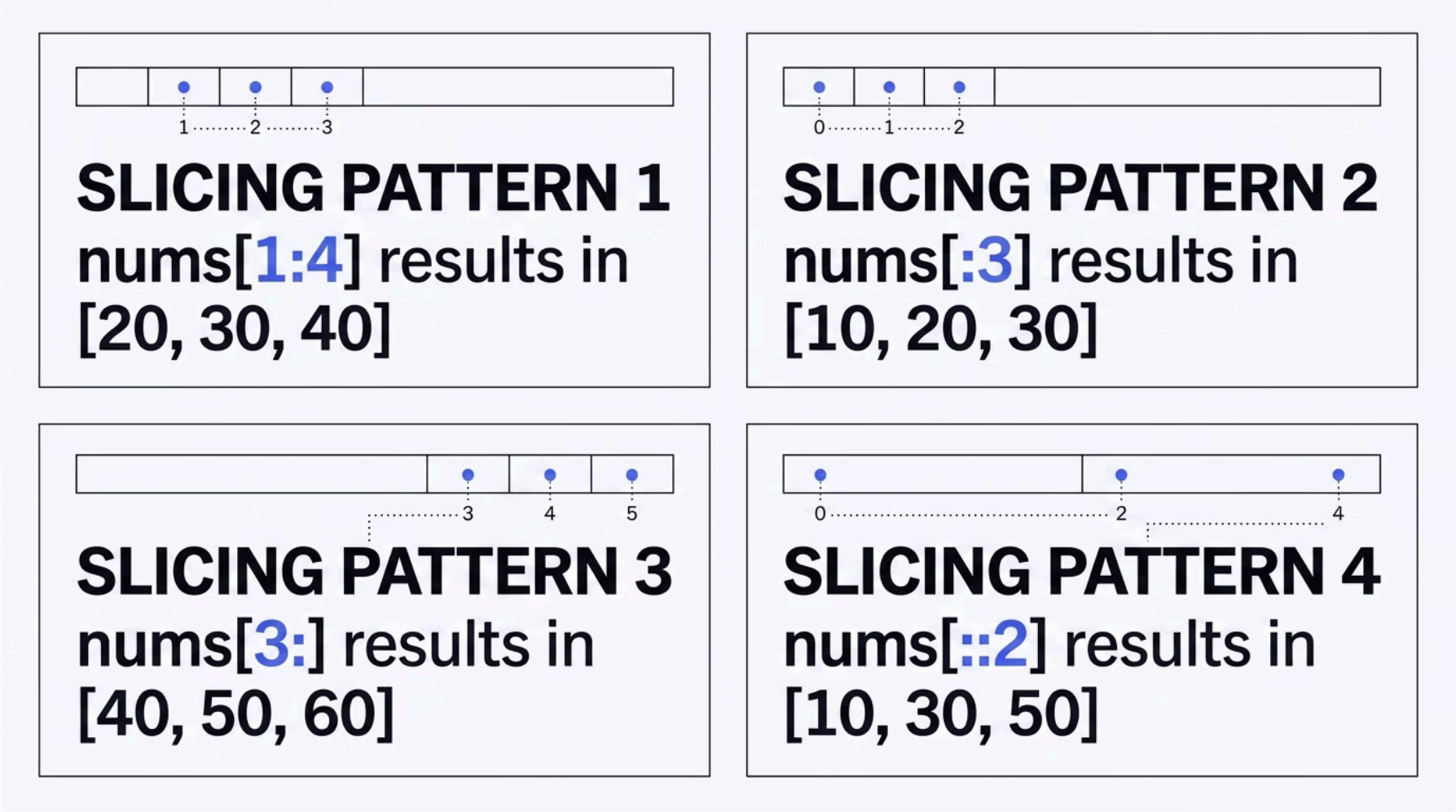
- `range(1, 11)` generates integers 1 through 10
- `i ** 2` computes the square of `i`
- Each square is appended to the growing list
- `sum(squares)` totals all 10 values: 1+4+9+...+100 = 385

Positive vs. Negative Indexing – Visual Summary



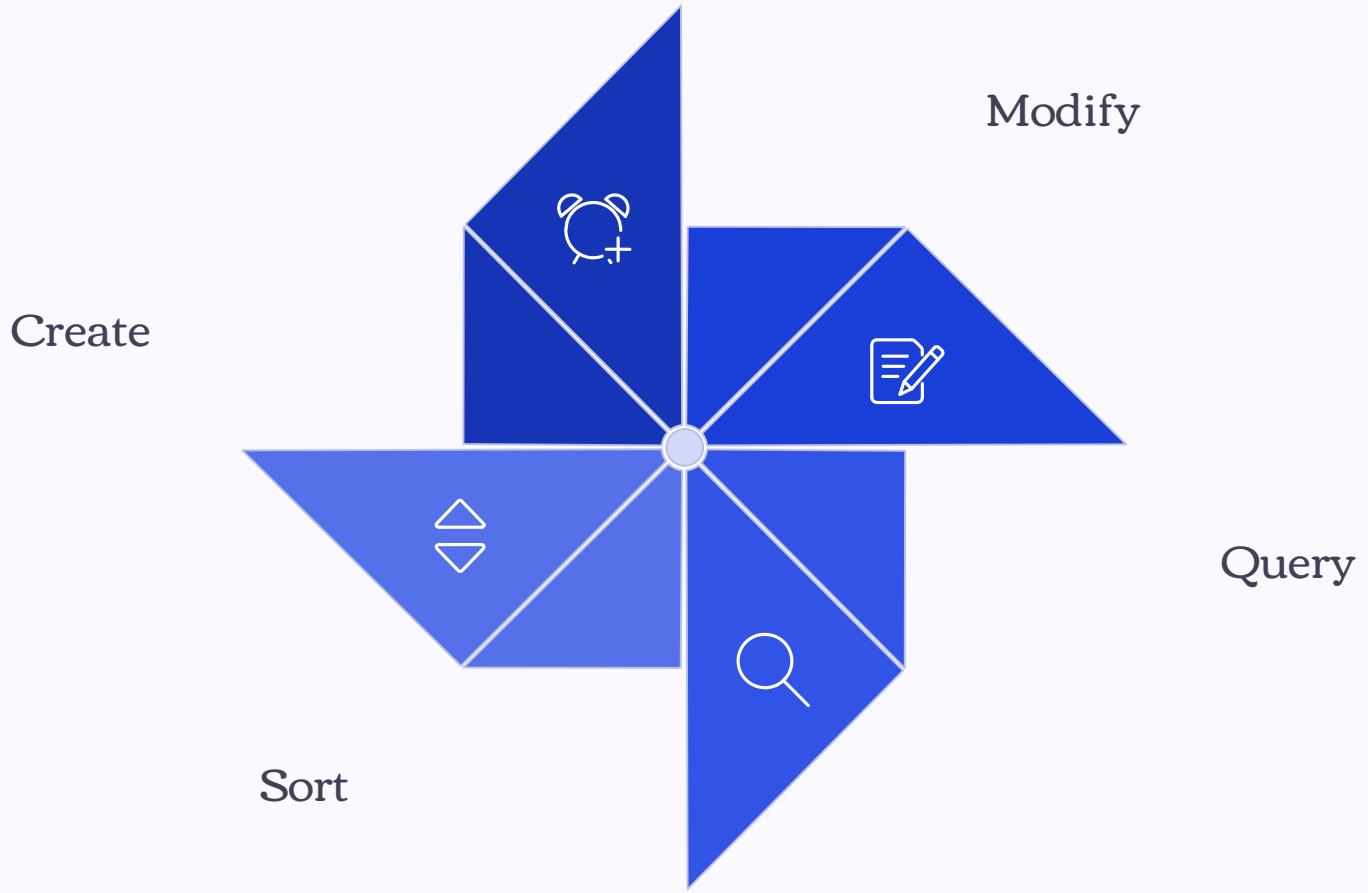
Every element in a list has two valid indices: a positive index counting from the front (starting at 0) and a negative index counting from the back (starting at −1). Both can be used interchangeably for access and slicing.

Slicing Patterns – Visual Summary



Slicing is one of Python's most powerful list features. Mastering the `start:stop:step` syntax allows you to extract any sub-sequence from a list without writing explicit loops.

List Methods – Workflow Overview



Understanding the full lifecycle of a list – from creation through modification, querying, and sorting – helps you choose the right method for each task and write cleaner, more efficient code.

sort() vs sorted()

lst.sort() – In-Place

Modifies the original list directly. Returns `None`. Use when you no longer need the original order.

```
nums = [3, 1, 4, 1, 5]
nums.sort()
# nums is now [1, 1, 3, 4, 5]
```

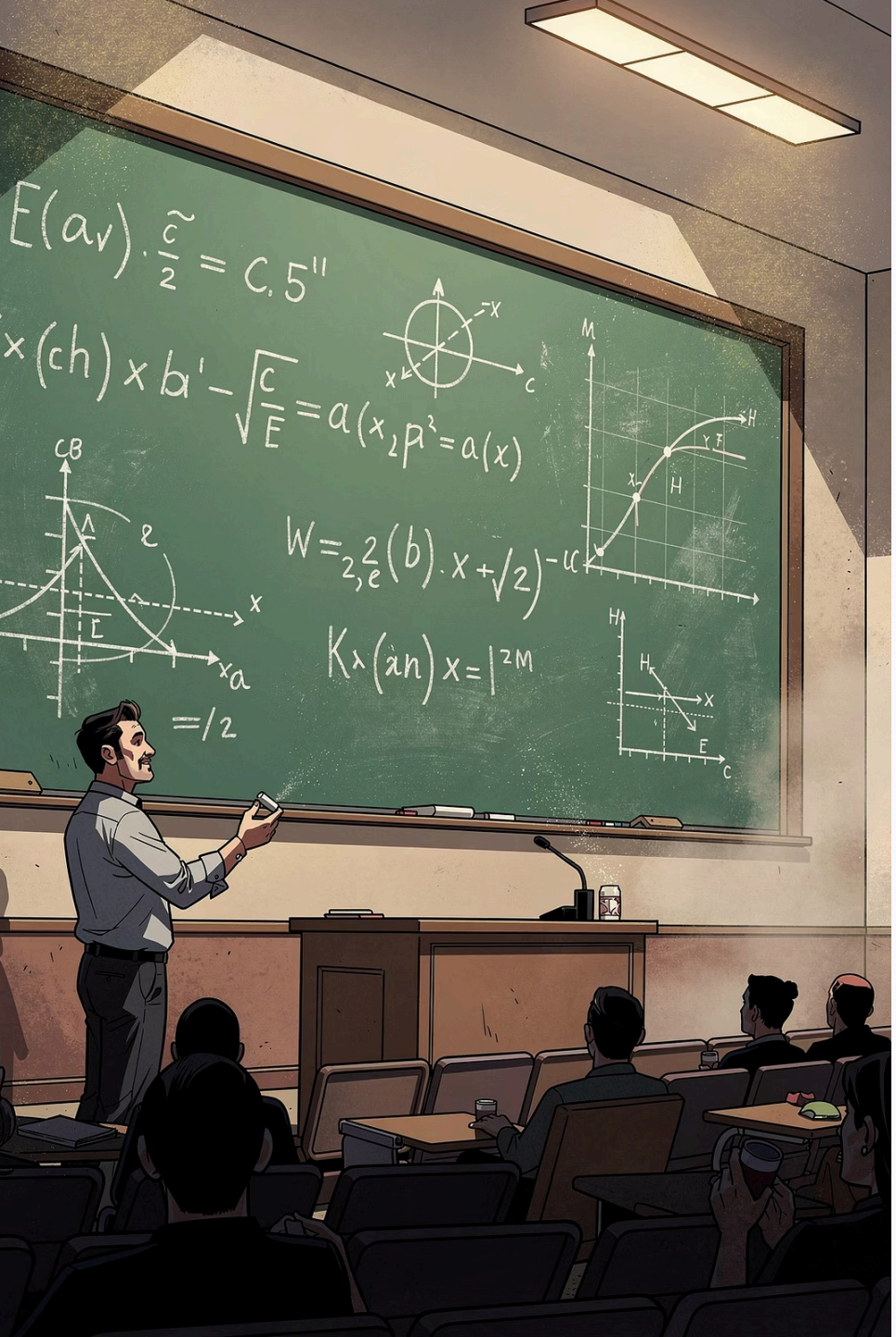
Add `reverse=True` to sort descending.

sorted(lst) – Returns New List

Returns a new sorted list without changing the original. Useful when you need both the original and sorted versions.

```
nums = [3, 1, 4, 1, 5]
s = sorted(nums)
# nums unchanged: [3, 1, 4, 1, 5]
# s = [1, 1, 3, 4, 5]
```

`sort()` is a list method; `sorted()` is a built-in function that works on any iterable, not just lists.



Common List Patterns in Mathematics

Accumulation

Initialize a variable to o, then loop through the list adding each element. Used for computing sums, products, and running totals.

Filtering

Loop through a list and collect elements that satisfy a condition (e.g., even numbers, values above a threshold) into a new list.

Transformation

Loop through a list and build a new list where each element is a function of the original (e.g., squares, absolute values, normalized values).

Building Lists Incrementally

A common pattern is to start with an empty list and use `append()` inside a loop to build it up. This is used in Exercise 4 to generate the squares list.

```
# General pattern
result = []
for item in source:
    result.append(transform(item))

# Concrete example: squares of 1–10
squares = []
for i in range(1, 11):
    squares.append(i ** 2)
# squares = [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

This pattern – empty list + loop + append – is fundamental. It appears in almost every real-world Python program that processes collections of data.

User Input into a List

Combining `input()`, type conversion, and `append()` lets you collect a variable number of values from the user into a list at runtime.

```
n = int(input("How many numbers? "))
nums = []
for i in range(n):
    x = float(input(f"Enter number {i+1}: "))
    nums.append(x)

print("You entered:", nums)
print("Sum:", sum(nums))
print("Average:", sum(nums) / len(nums))
```

Sample Run

```
How many numbers? 3
Enter number 1: 10
Enter number 2: 20
Enter number 3: 30
You entered: [10.0, 20.0, 30.0]
Sum: 60.0
Average: 20.0
```

Key Concepts

- `int(input(...))` reads the count
- `float(input(...))` reads each number
- `range(n)` controls the loop count
- The list grows dynamically with each `append()`

Nested Loops with Lists

You can use nested loops to process two-dimensional data stored as a list of lists (a matrix). Each inner list represents one row.

```
matrix = [[1, 2, 3],
          [4, 5, 6],
          [7, 8, 9]]

for row in matrix:
    for val in row:
        print(val, end=" ")
    print()  # newline after each row
```

Output

```
1 2 3
4 5 6
7 8 9
```

Access by Index

Access individual elements with two indices: `matrix[row][col]`. For example, `matrix[1][2]` → `6` (row 1, column 2).

ⓘ Nested lists (lists of lists) are a simple way to represent matrices and tables before learning NumPy arrays in later courses.

Checking Membership in a List

Python's `in` and `not in` operators let you test whether a value exists in a list without writing a loop.

in Operator

```
fruits = ["apple", "mango", "cherry"]
print("mango" in fruits)   # True
print("banana" in fruits)  # False
```

not in Operator

```
scores = [85, 90, 78]
if 100 not in scores:
    print("No perfect score yet")
```

Practical Use

Use membership tests to guard against duplicates before appending, or to validate user input against a list of allowed values.

List Concatenation and Repetition

Concatenation with +

Combine two lists into a new list using the `+` operator. The original lists are unchanged.

```
a = [1, 2, 3]
b = [4, 5, 6]
c = a + b
# c = [1, 2, 3, 4, 5, 6]
```

Repetition with *

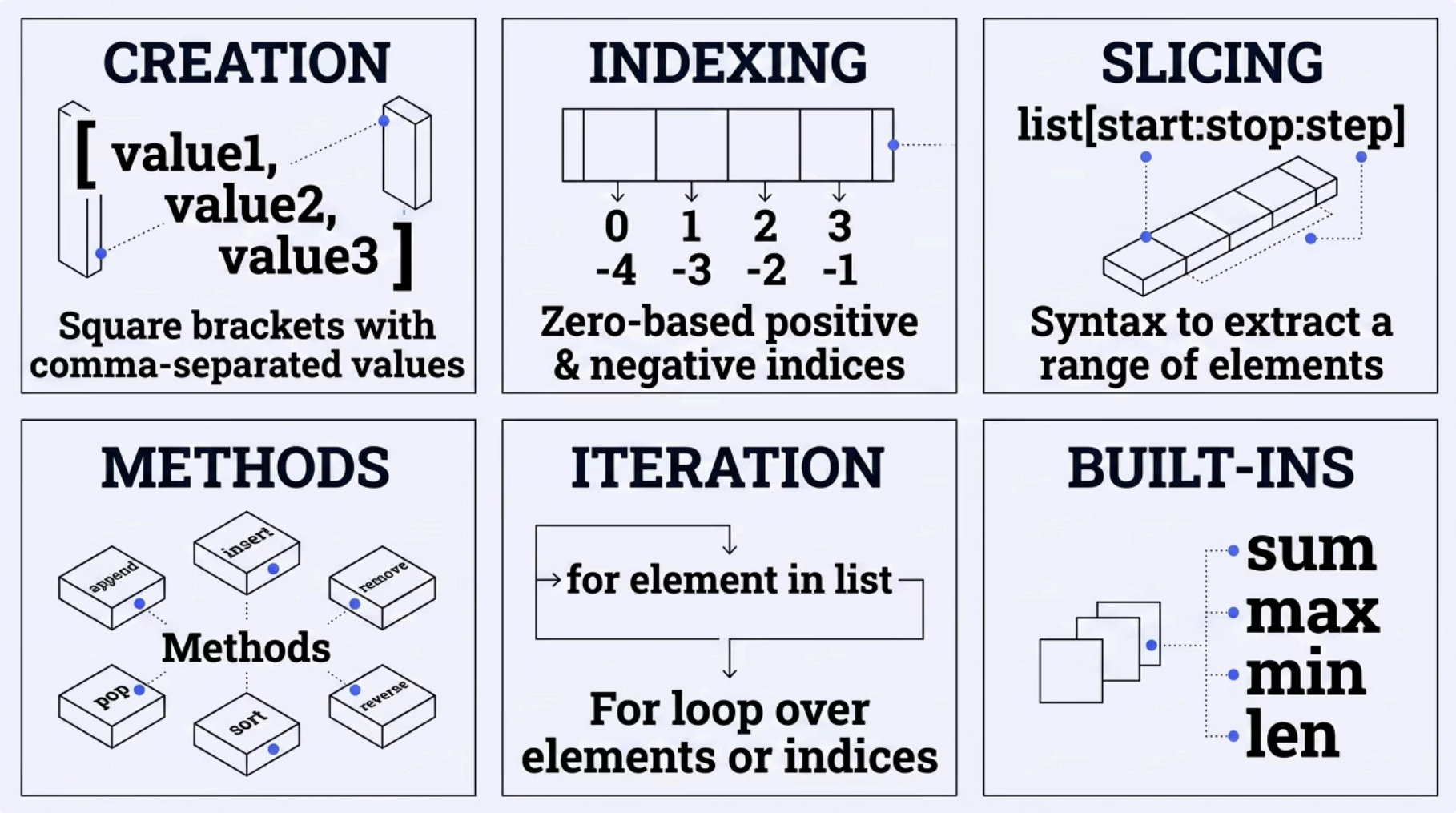
Repeat a list a given number of times using the `*` operator. Useful for initializing lists with a default value.

```
zeros = [0] * 5
# zeros = [0, 0, 0, 0, 0]

pattern = [1, 2] * 3
# pattern = [1, 2, 1, 2, 1, 2]
```

These operators create **new** lists. To add elements to an existing list, use `append()` or `extend()` instead.

Key Concepts at a Glance



These six pillars form the complete foundation of list programming in Python. Mastering all six enables you to handle virtually any data management task in mathematics and beyond.

Chapter 6 – Full Summary

Lists store multiple ordered values in one variable

Created with `[ ]`; elements can be of any type. Access with positive indices (from 0) or negative indices (from −1). Raises `IndexError` if index is out of range.

Slicing extracts sub-ranges without loops

Syntax: `list[start:stop:step]`. Omitting `start` or `stop` defaults to the list boundaries. Never raises `IndexError`.

Lists are mutable – use methods to modify them

Key methods: `append`, `insert`, `remove`, `pop`, `sort`, `reverse`, `index`, `count`.

Iterate with for; compute with built-in functions

Loop directly over elements or use index-based `range(len(lst))`. Use `sum()`, `max()`, `min()`, `len()` for fast numeric operations.

Ready for the Next Chapter

You have now mastered the fundamentals of Python list data structures – the most widely used tool for mathematical data management in Python. With lists, you can store, access, modify, iterate, and compute over collections of values efficiently.

What You Learned

- Create and index lists (positive & negative)
- Slice sub-ranges with `start:stop:step`
- Modify lists with 8 key methods
- Iterate with `for` and use `sum/max/min/len`
- Apply lists to statistical calculations

Coming Up Next

The next chapter will introduce additional data structures such as tuples, dictionaries, and sets – expanding your toolkit for handling complex mathematical and real-world data.

CLO 4 achieved: Students can store and compute mathematical data using Python list structures.